Persistencia offline-first

Desarrollo de Aplicaciones Móviles
Guía Práctica de Laboratorio N.º 11

Universidad Peruana Unión
EP Ingeniería de Sistemas · Ciclo VI · Semestre 2026-2

Estudiante: Jorge Armando Quispe Huaman
Fecha: 21 de octubre de 2026
Proyecto: PharmaMobil (Android/KMP) + PharmaSoft (API REST)

Resultado demostrado
Producto creado en modo avión: **A_Offline_S11_1531**.
ID local **-1**, pendiente **1** → ID del servidor **63**, pendiente **0**.
Postman: **200 OK** y una sola coincidencia del producto.

Validación: 76 pruebas aprobadas, 0 fallos; APK Android compilado. La app reabrió y listó con modo avión y con PharmaSoft detenido.

Alcance acordado: Android. iOS excluido por indicación del usuario. Postman sustituye Swagger para las evidencias del servidor.

Repositorio: github.com/123456jor/pharmaMobile-
Rama local: feature/offline-first-quispe
Los commits de esta práctica están guardados localmente; su publicación requiere la confirmación final solicitada en el encargo.

Implementación realizada

01 · Base local
SQLDelight 2.1.0 genera PharmaDb. Android usa AndroidSqliteDriver y crea pharma.db. Koin registra el driver, la base, ProductoLocal y ProductoRepositorioOffline.

02 · Esquema conservando el CRUD
producto contiene id, nombre, precio, stock y pendiente. Se conservan estado, categoriaId y categoriaNombre para mantener categorías y bajas lógicas del proyecto existente.

03 · Una fuente para la pantalla
ProductoLocal observa consultas SQLDelight mediante Flow. ProductoViewModel recoge ese Flow; no consulta manualmente la lista después de registrar, editar o eliminar.

04 · Subir y después bajar
Se guardan primero las altas pendientes con ID negativo. La sincronización serializada envía POST, confirma la sustitución local por ID positivo y luego descarga todas las páginas del servidor. El reemplazo transaccional conserva los pendientes.

05 · Sesión al reabrir sin red
Se recupera únicamente una sesión previamente autenticada, cifrada y vigente. Un 401 o el vencimiento impiden usar esa sesión. El primer login y la renovación necesitan conexión.

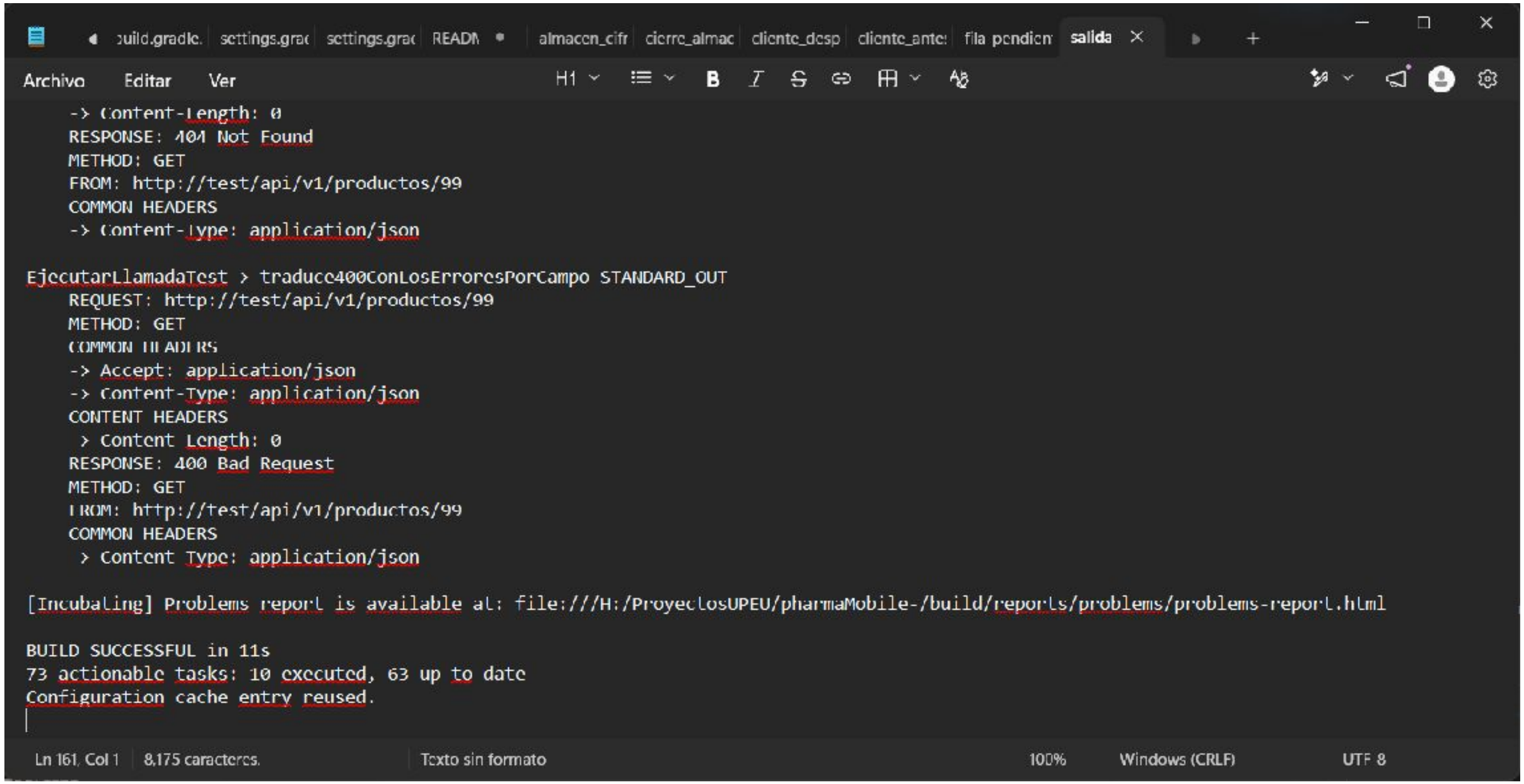
06 · Acciones conservadas
Registrar y consultar funcionan sin red. Editar y eliminar se mantienen con conexión y solo para productos sincronizados, según la decisión expresa del usuario.

Código, instrucciones de ejecución y adaptaciones: README.md, sección **Persistencia offline-first**. Verificación detallada de cada requisito: evidencias/checklist.md.

Pruebas automatizadas y compilación

76 pruebas, 0 fallos, 0 errores, 0 omitidas. Ocho pruebas del repositorio offline y tres de sesión offline se suman a la suite previa. La misma ejecución ensambló el APK Android.

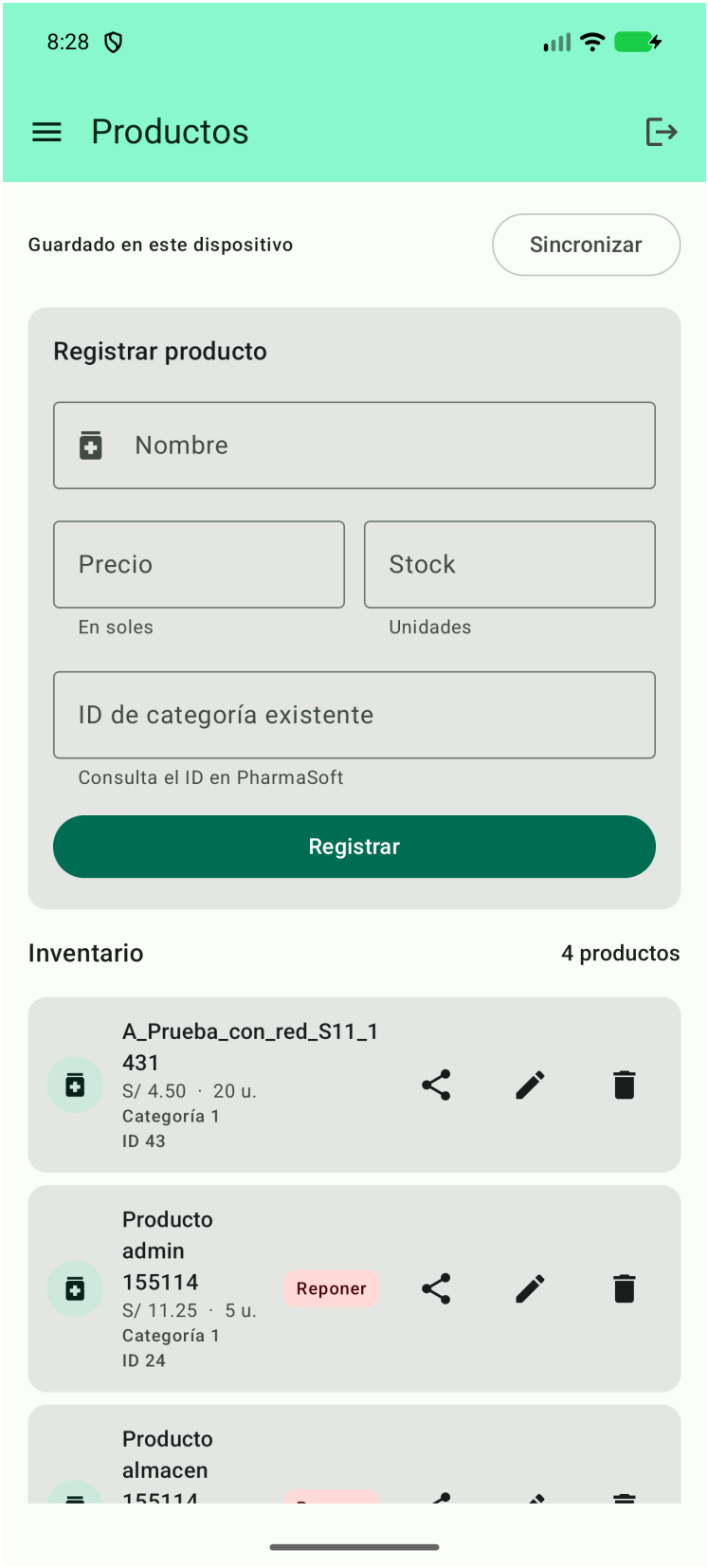
Vista real del final de salida-pruebas.txt en Bloc de notas. Las respuestas 400/404 visibles corresponden a casos controlados de manejo de errores. El total se verificó con los resultados XML; se conserva también el reporte HTML de las 17 suites.



Original: evidencias/capturas/09-pruebas-build-successful.jpg

01 · Inventario con conexión

La app sincronizó y mostró cuatro productos activos. Se verificó el alta con red A_Prueba_con_red_S11_1431 con ID 43 y después se cerró la app por completo.

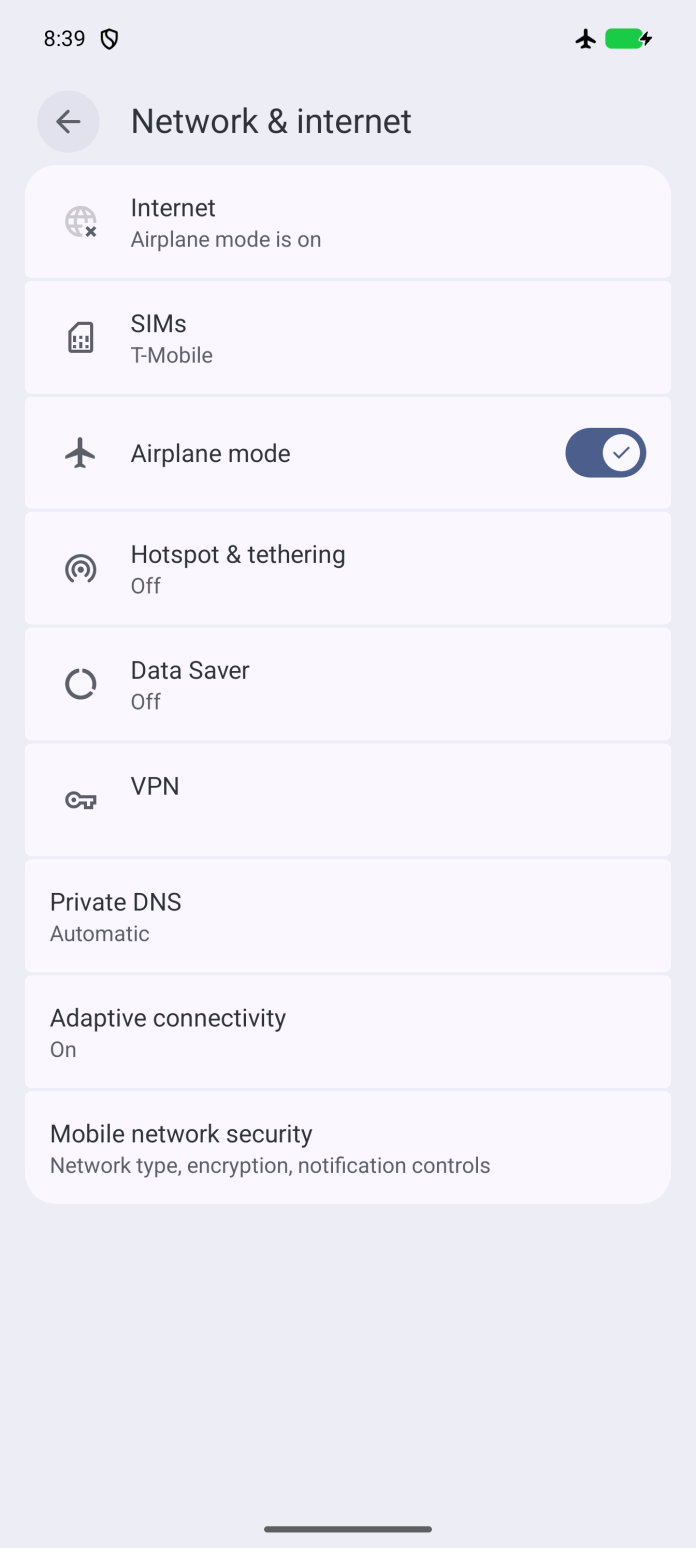


Original: evidencias/capturas/01-con-red-sincronizado.png

02 · Modo avión activado

El emulador permite activar modo avión. Durante el ciclo también se desactivaron Wi-Fi y datos para asegurar la ausencia de conexión.

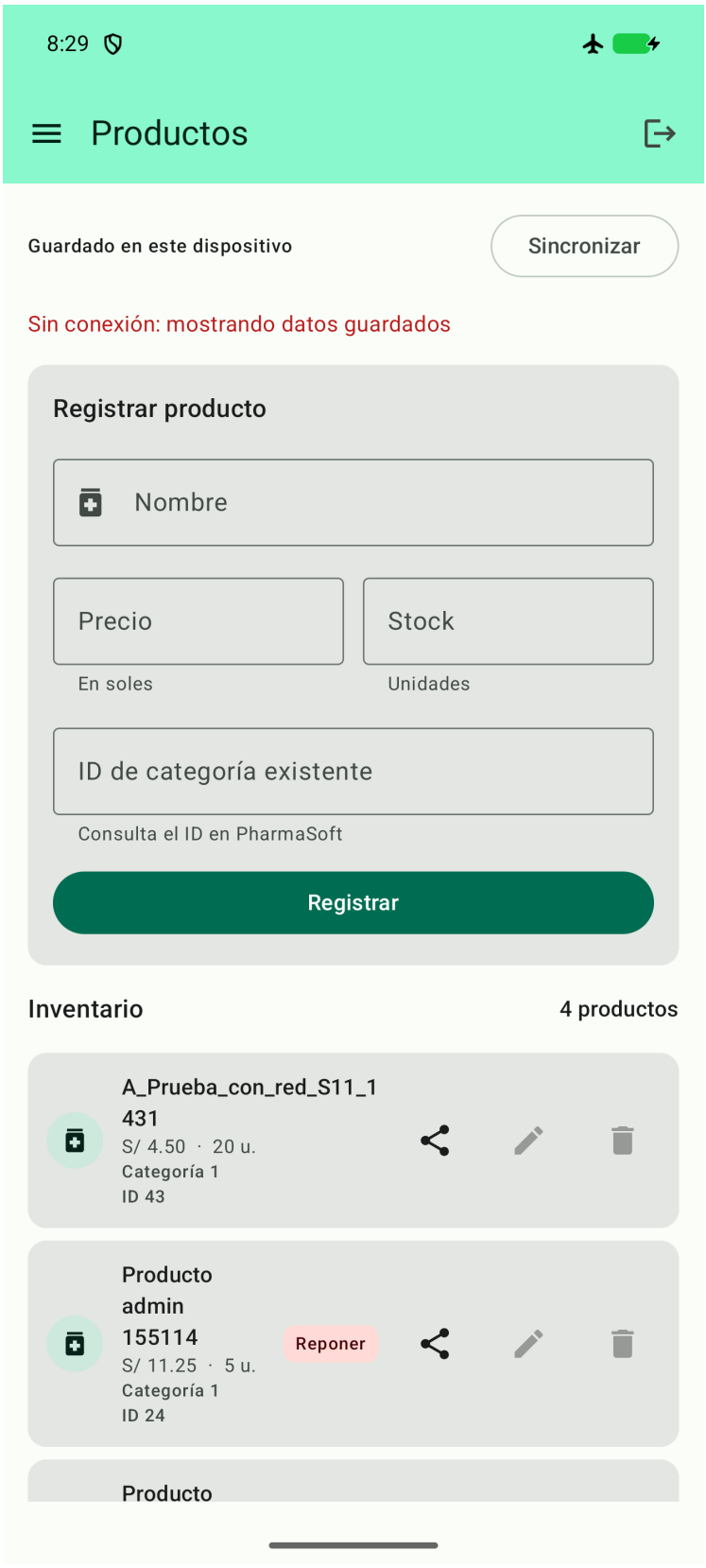
Esta captura de Ajustes se repitió después del ciclo porque la inicial quedó durante una animación. El modo avión del ciclo original está documentado también en modo-avion.txt y en la barra de las capturas 03 y 04.



Original: evidencias/capturas/02-modo-avion-activo.png

03 · Reapertura sin conexión

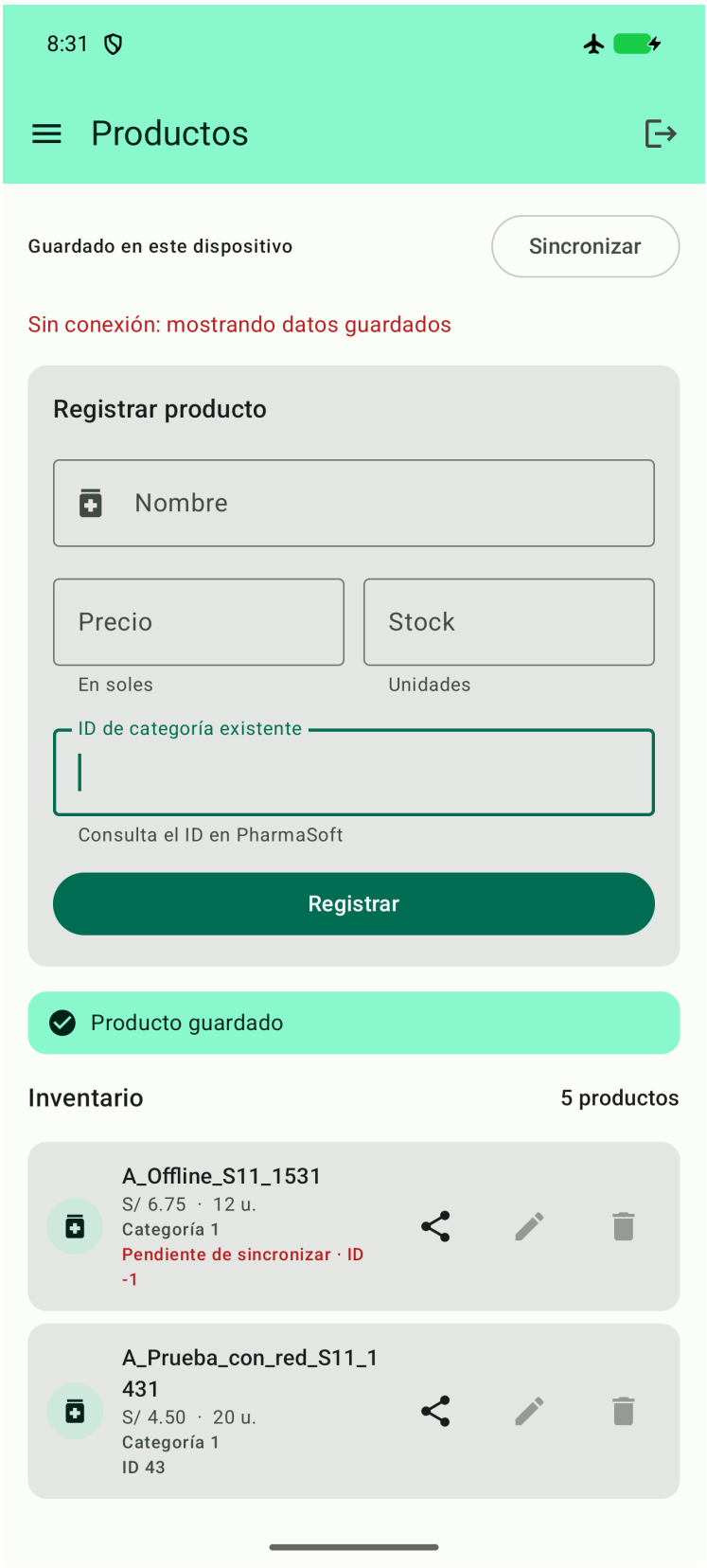
Tras cerrar y reabrir con modo avión, aparecieron los mismos cuatro productos. El aviso visible indica: Sin conexión: mostrando datos guardados.



Original: evidencias/capturas/03-listado-sin-red-android.png

04 · Registro sin conexión

A_Offline_S11_1531 apareció en el inventario sin red. La tarjeta muestra Pendiente de sincronizar e ID -1. El listado pasó de cuatro a cinco productos.

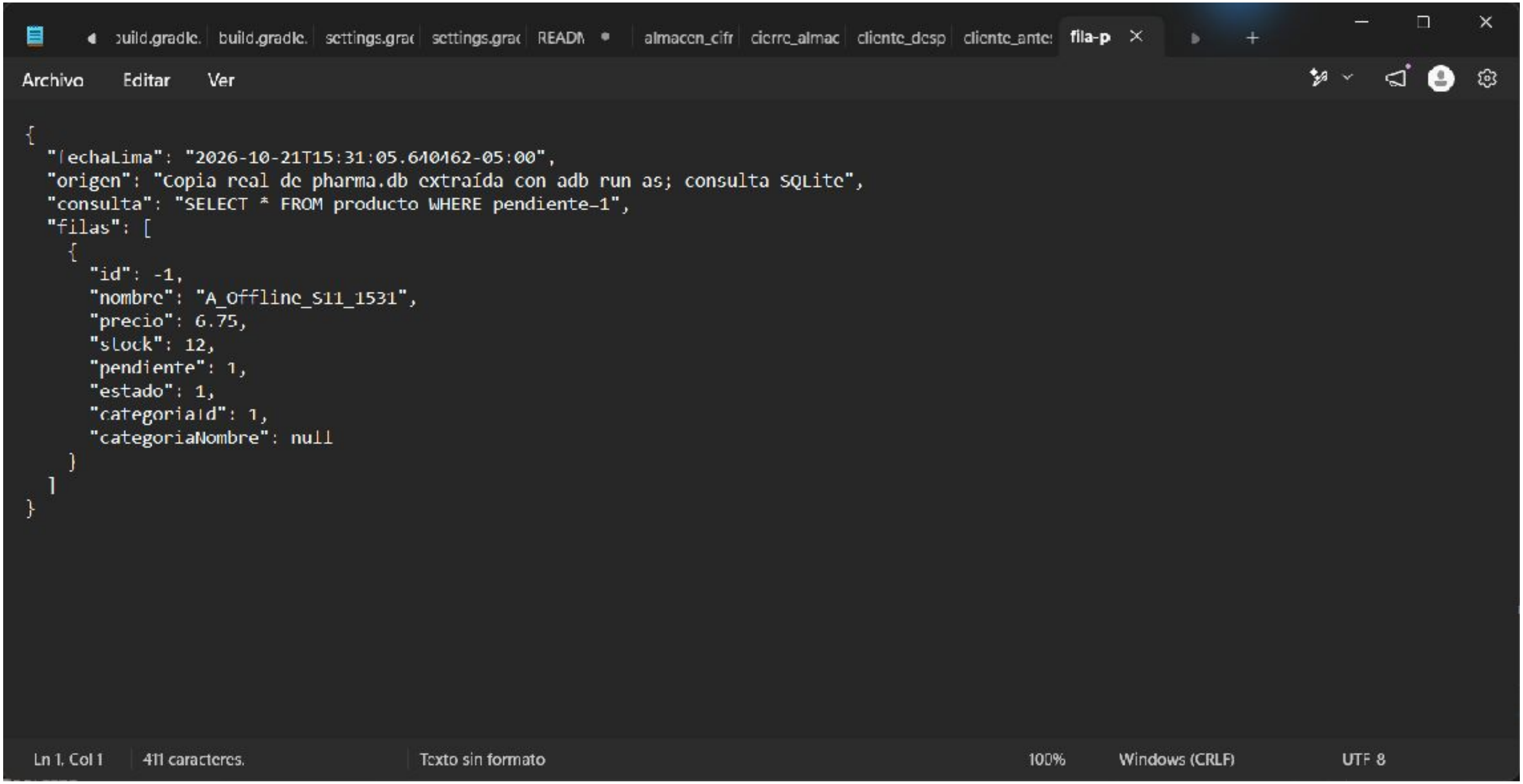


Original: evidencias/capturas/04-registro-sin-red.png

05 · Fila pendiente en SQLite

Consulta real sobre una copia de pharma.db extraída de la app: SELECT * FROM producto WHERE pendiente=1. Se verificó precio 6.75, stock 12, categoría 1 e ID -1.

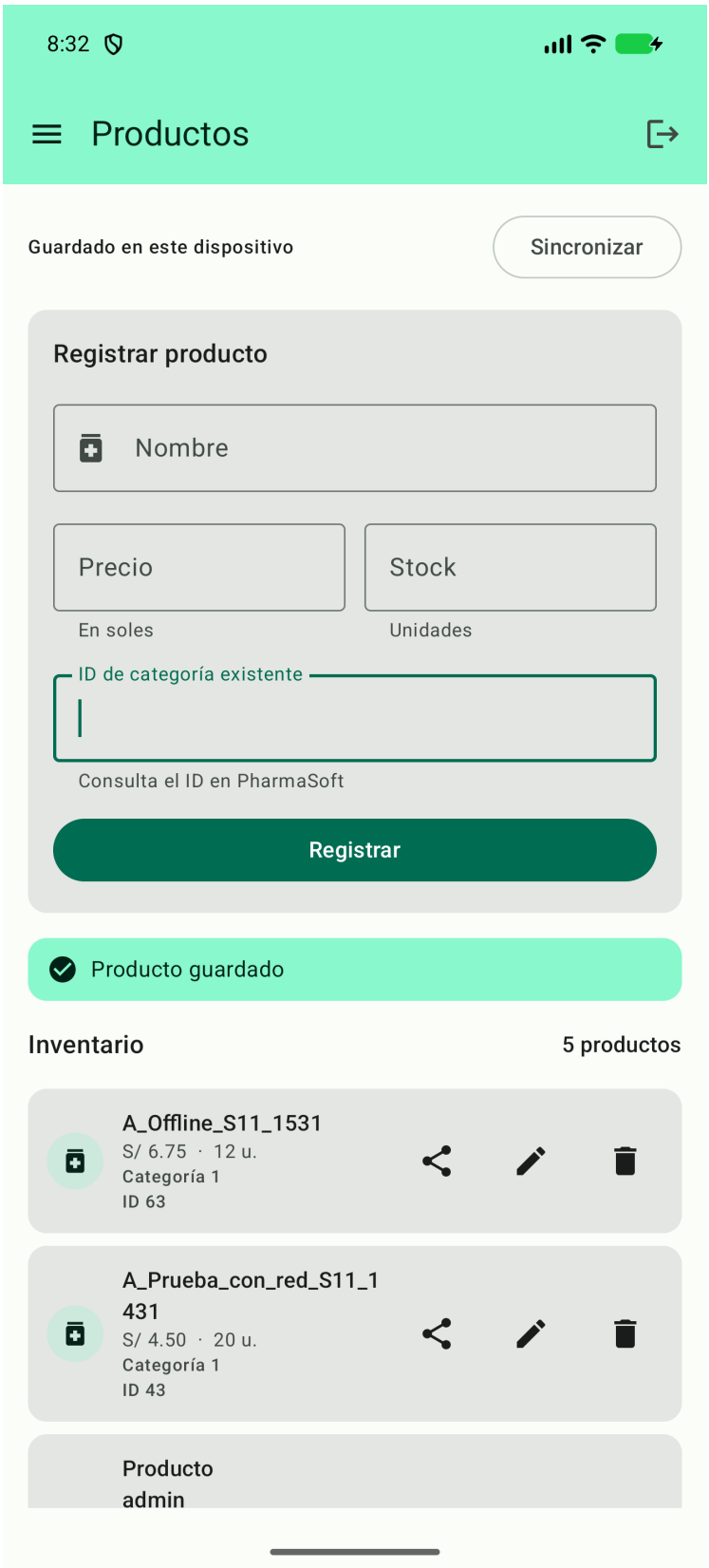
La captura muestra el JSON de esa consulta en Bloc de notas. Es un visor del resultado SQLite guardado; no se presenta como Database Inspector ni como respuesta HTTP. Original verificable: evidencias/pruebas/fila-pendiente.json.



Original: evidencias/capturas/05-fila-pendiente.jpg

06 · Reconexión y sincronización

Se desactivó modo avión, se recuperó la red y se pulsó Sincronizar. El producto conservó sus datos y quedó una sola vez con ID 63. SQLite confirmó pendiente=0 y cero pendientes totales.

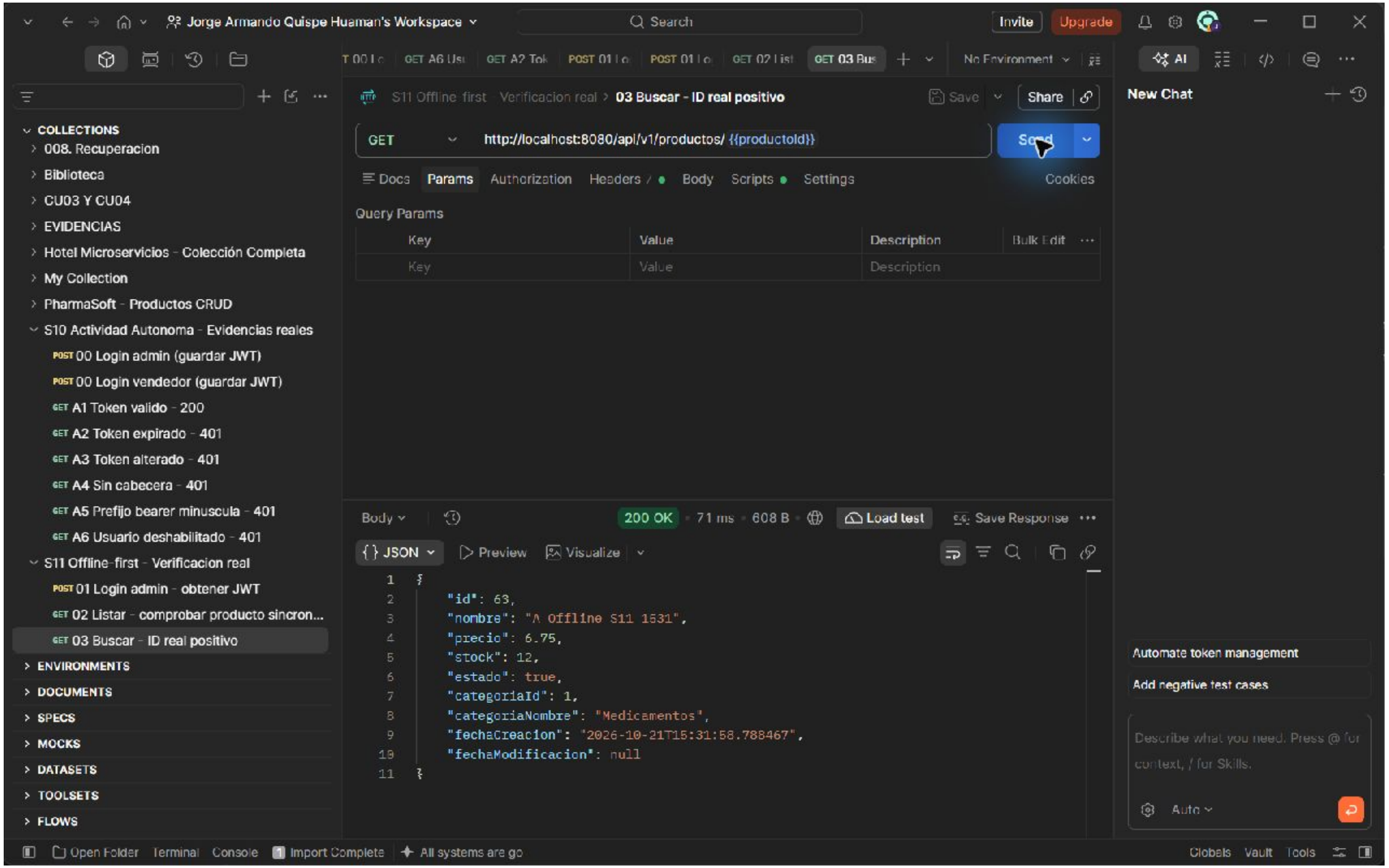


Original: evidencias/capturas/06-reconexion-sincronizar.png

07 · Producto confirmado en Postman

GET /api/v1/productos/{{productoId}} devolvió 200 OK. El cuerpo del servidor muestra ID 63, nombre A_Offline_S11_1531, precio 6.75, stock 12 y categoría Medicamentos.

Se utiliza Postman en lugar de Swagger por instrucción expresa del usuario. La variable productoId se obtuvo de la consulta del listado; el registro se creó desde Android en modo avión.

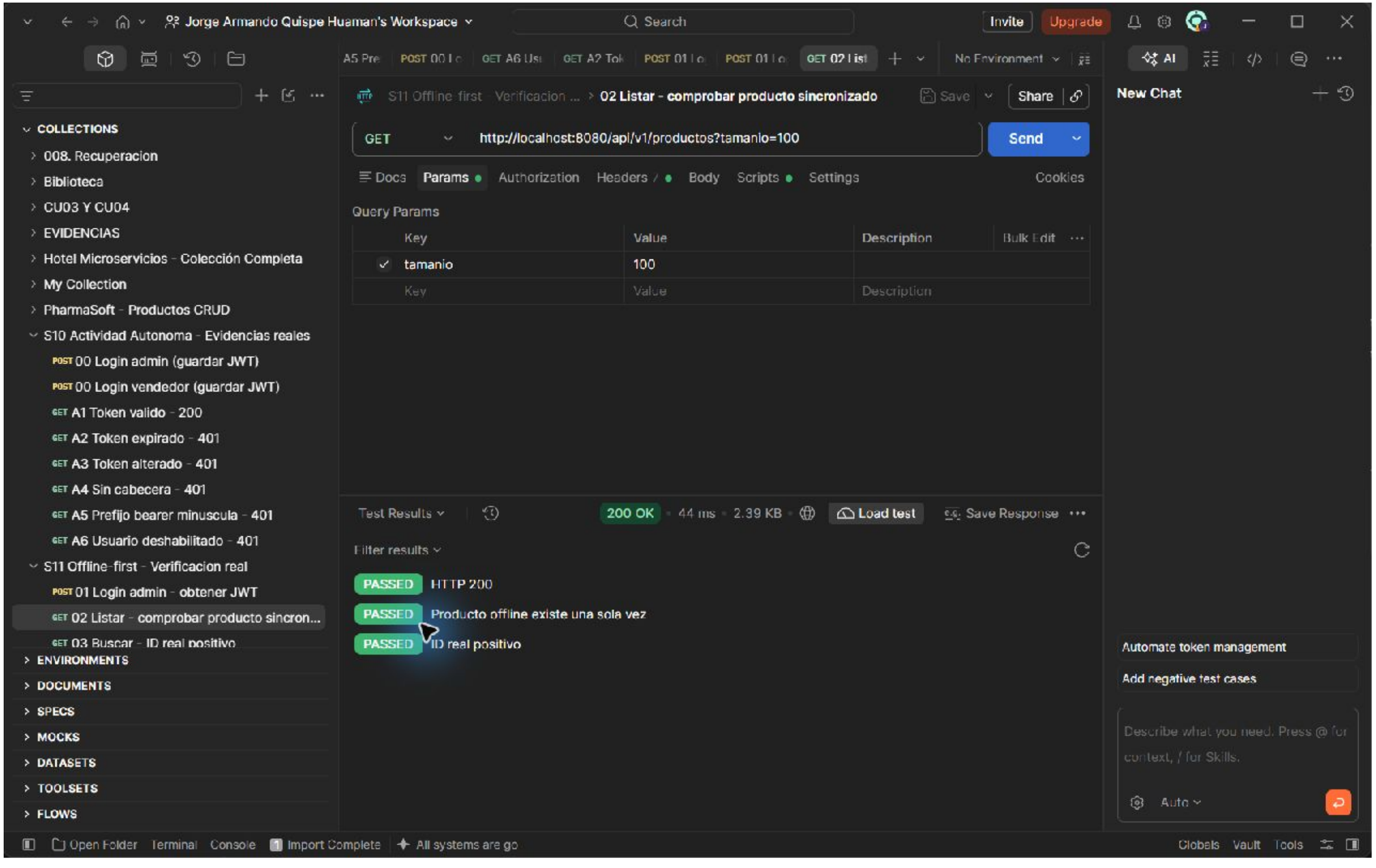


Original: evidencias/capturas/07-postman-producto-id-real.jpg

08 · Comprobaciones de Postman

El GET del listado aprobó tres comprobaciones: HTTP 200, Producto offline existe una sola vez e ID real positivo. Esto complementa la captura del cuerpo del producto.

Colección reproducible: postman/S11-Offline-first.postman_collection.json. Orden: 01 Login, 02 Listar, 03 Buscar. El archivo exportado no contiene JWT.

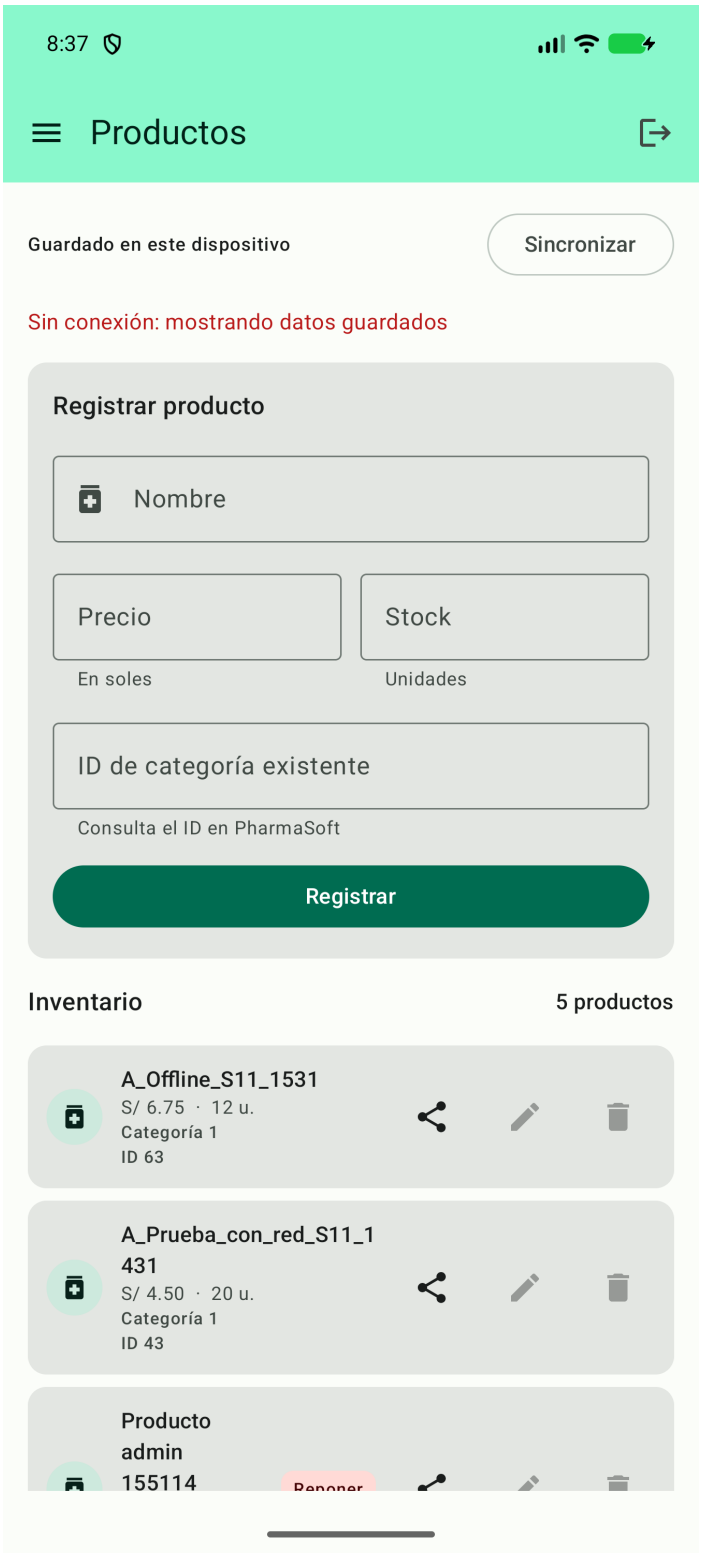


Original: evidencias/capturas/07b-postman-listado-pruebas.jpg

09 · PharmaSoft apagado

Con la red activa y PharmaSoft detenido, se cerró y reabrió la app. Mostró cinco productos guardados y el aviso de falta de conexión.

Se comprobó que el puerto 8080 no tenía un proceso escuchando. Después de la captura se reinició PharmaSoft: health respondió 200 a las 15:38, hora de Lima. Oracle/Docker se mantuvo activo.

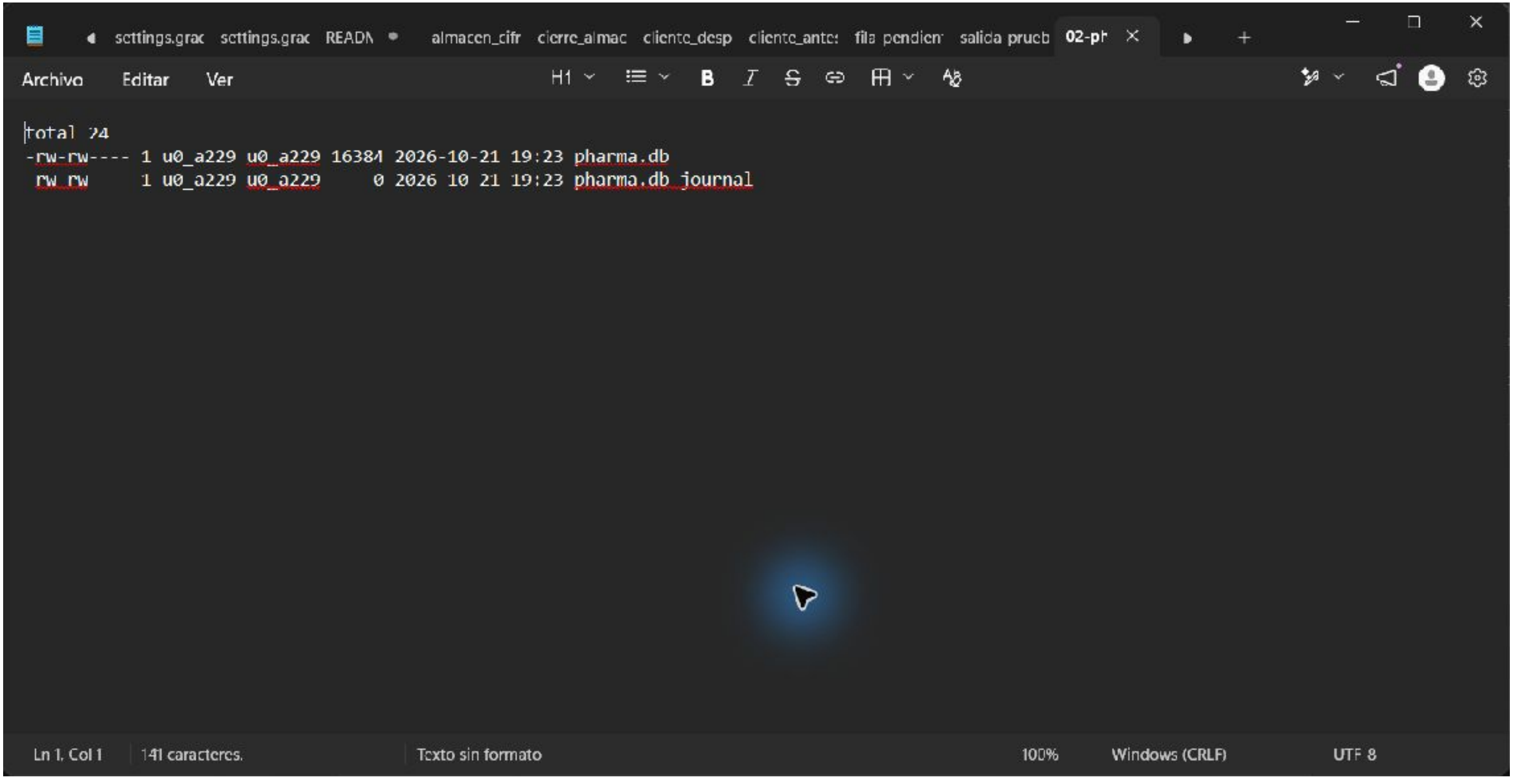


Original: evidencias/capturas/08-backend-apagado-listado-android.png

10 · Archivo pharma.db creado

El control del driver confirmó que existe pharma.db dentro de databases de la aplicación Android. Se conserva la salida original de adb run-as ls -l databases.

Captura auténtica del archivo de salida 02-pharma-db.txt abierto en Bloc de notas. Es una vista del resultado de ADB, no una terminal simulada ni Device Explorer.



Original: evidencias/capturas/10-archivo-pharma-db.jpg

Lista de cotejo

N.º	Criterio	Resultado
1	Esquema SQLDelight compila	Generación de PharmaDb y compilación correctas.
2	Driver registrado	Android y JDBC de pruebas verificados. iOS excluido.
3	Capa local sin Ktor ni DTO	ProductoLocal trabaja con SQL y dominio.
4	Contrato observar / sincronizar	Implementado, inyectado y probado.
5	ViewModel con Flow	Sin recarga manual después de mutaciones.
6	Lista con backend apagado	Reapertura y captura real con cinco productos.
7	Pendiente marcado	ID -1 y pendiente=1 en UI y consulta SQLite.
8	Sube antes de bajar	Prueba de orden POST antes de GET aprobada.
9	Producto existe en PharmaSoft	Postman 200, ID 63, una sola coincidencia.
10	Al menos tres pruebas	Ocho del repositorio; 76 totales aprobadas.

El detalle de E1-E3, P1-P6, T1-T4 y D1-D6, con archivos, verificaciones y evidencias, está en **evidencias/checklist.md**. Los puntos de iOS no se reclaman completados.

Adaptaciones y trazabilidad

Rama: develop no existe. Se creó feature/offline-first-quispe desde la sesión 10 con autorización del usuario.

D-1: JDBC se ubica en androidHostTest mediante una fábrica expect/actual. La lógica de las pruebas está en commonTest, con una base nueva por caso.

D-2: el algoritmo literal dejó dos filas. Se conservó la prueba fallida y se corrigió con borrarPorId y confirmación transaccional solo después del POST exitoso.

D-3 y D-4: alias ProductoFila para el modelo generado; edición y eliminación online conservadas por decisión expresa.

Alcance: Android, sin implementación ni pruebas iOS. Postman sustituye Swagger por indicación expresa.

Commits realizados durante la implementación

dbcc133 feat(db): agrega SQLDelight 2.1.0 y esquema de producto

010de22 feat(db): registra driver Android y PharmaDb en Koin

963e2a6 feat(local): observa productos y conserva pendientes en transacciones

2997a2e feat(data): agrega repositorio offline con subida antes de descarga

1e68494 test: cubre offline-first y corrige duplicacion confirmada de pendientes

147d3b6 feat(ui): observa el Flow local y sincroniza sin recargas manuales

217c036 fix(auth): restaura sesion cifrada vigente al abrir sin red

La documentación, las capturas originales, el reporte de pruebas y este PDF se guardan en el proyecto. El README incluye cómo reproducir el ciclo y la limitación de idempotencia ante un POST confirmado en el servidor cuya respuesta se pierda.